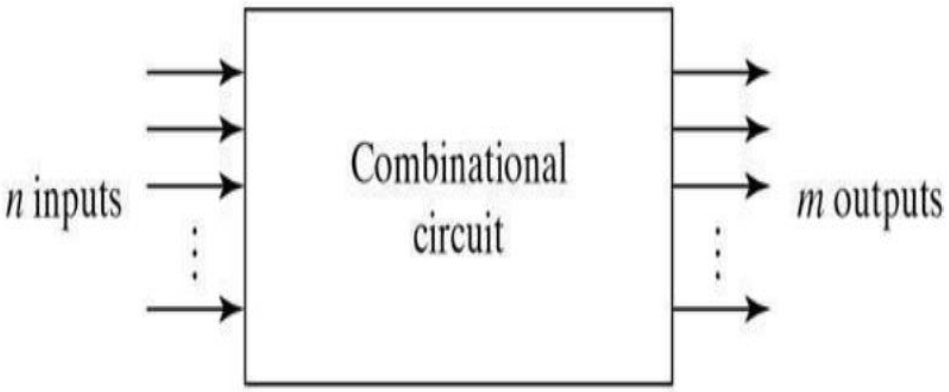
UNIT II

Combinational Circuits Design and Analysis

Combinational Logic:

• Logic circuits for digital systems may be combinational or sequential.

• A combinational circuit consists of input variables, logic gates, and output variables.



For n input variables, there are 2n possible combinations of binary input variables. For each possible input combination, there is one and only one possible output combination. A combinational circuit can be described by 'm' Boolean functions one for each output variables.

A Systematic Approach to Combinational Logic Design:

The Synthesis of combinational logic starts with a problem statement and progresses through a series of steps, terminating in final circuit design. The combinational logic design steps are as follows:

1. Develop a statement describing the problem to be solved.

2. Based on the problem statement, construct a truth table that clearly establishes the relationship between the input and output variables.

3. Use k-Maps or QMC techniques to simplify the functions in deriving the output equations. Output equations be expressed as either SOP or POS. The best solution will require the fewer gates and gate inputs.

4. Draw the final logic diagram.

Adders:

Adders are divided into two groups: half adders and full adders. Full adders are used toadd three bits where one of them is the carry from the preceding adder. They have twooutputs: the sum and the carry to the next stage. In half adders, only two inputs are considered as operands; carry inputs are ignored. The truth table of a full adder is given in Table 2-1. Two of the input variables, denoted by x and y, represent the two significant bits to be added. The third input, C k-1 represents the carry from the previous lower significant position.

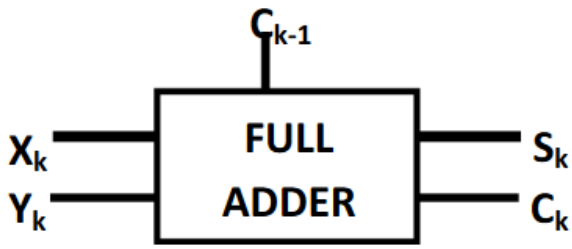


Figure 2-1. Block diagram of a full adder.

X_k	Y_k	C_{k-1}	S_k	C_k
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 2-1. Truth table of a full adder (X+Y)

Substractors:

Subtractors are similar to adders. There are full subtractors with three inputs one of which is the borrow to the preceding subtractor. The two outputs are the difference and the borrow from the succeeding unit. Half subtractors do not have a borrow input. Figure 4-2shows a block diagram of a full subtractor, and Table 4-2 gives the truth table of a full subtractor.

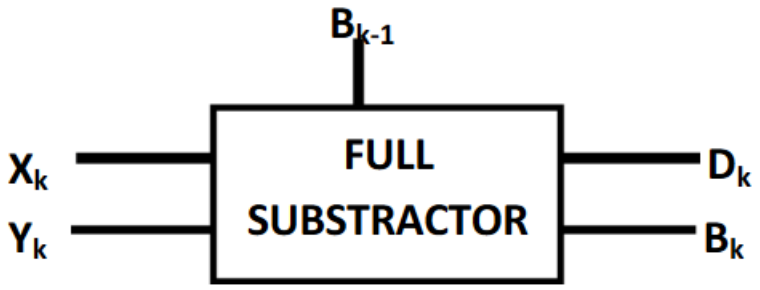


Figure 2-2. Block diagram of a full substractor

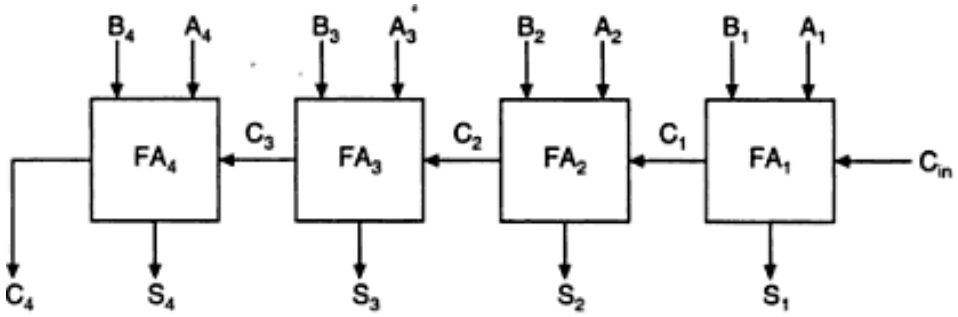
X_k	Y_k	B_{k-1}	B_k	D_k
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Table 2-2. Truth table of a full substractor

Binary Parallel Adder:

A binary parallel adder is a digital circuit that adds two binary numbers in parallel form and produces the arithmetic sum of those numbers in parallel form. It consists of full adders connected in a chain, with the output carry from each full-adder connected to the input carry of the next full-adder in the chain.

The interconnection of four full-adder (FA) circuits to provide a 4-bit parallel adder. The augends bits of A and addend bits of B are designated by subscript numbers from right to left, with subscript 1 denoting the lower –order bit. The carries are connected in a chain through the full-adders. The input carry to the adder is Cin and the output carry is C4. The S output generates the required sum bits. When the 4-bit full-adder circuit is enclosed within an IC package, it has four terminals for the augends bits, four terminals for the addend bits, four terminals for the sum bits, and two terminals for the input and output carries. AN n-bit parallel adder requires n-full adders. It can be constructed from 4-bit, 2-bit and 1-bit full adder ICs by cascading several packages. The output carry from one package must be connected to the input carry of the one with the next higher –order bits. The 4-bit full adder is a typical example of an MSI function.



Logic diagram of a 4-bit binary parallel adder.

Ripple carry adder:

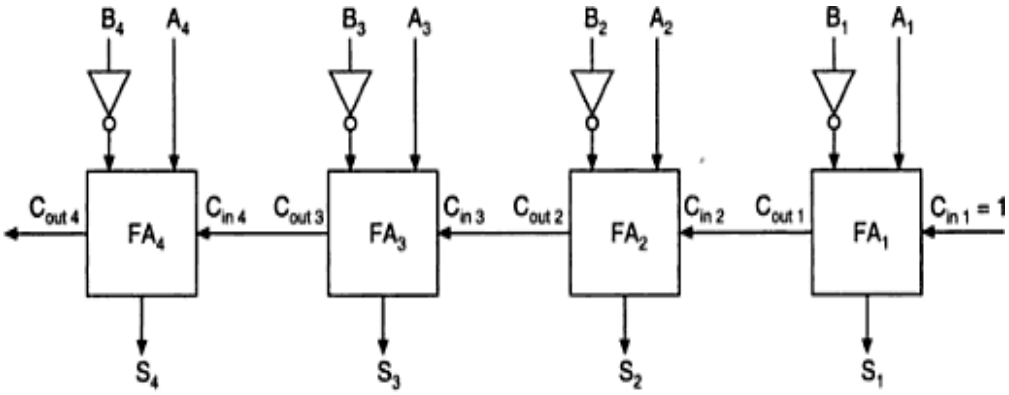
In the parallel adder, the carry –out of each stage is connected to the carry-in of the next stage. The sum and carry-out bits of any stage cannot be produced, until sometime after the carry-in of that stage occurs. This is due to the propagation delays in the logic circuitry, which lead to a time delay in the addition process. The carry propagation delay for each full- adder is the time between the application of the carry-in and the occurrence of the carry-out.

The 4-bit parallel adder, the sum (S1) and carry-out (C1) bits given by FA1 are not valid, until after the propagation delay of FA1. Similarly, the sum S2 and carry-out (C2) bits given by FA2 are not valid until after the cumulative propagation delay of two full adders (FA1 and FA2) , and so on. At each stage, the sum bit is not valid until after the carry bits in all the preceding stages are valid. Carry bits must propagate or ripple through all stages before the most significant sum bit is valid. Thus, the total sum (the parallel output) is not valid until after the cumulative delay of all the adders.

The parallel adder in which the carry-out of each full-adder is the carry-in to the next most significant adder is called a ripple carry adder. The greater the number of bits that a ripple carry adder must add, the greater the time required for it to perform a valid addition. If two numbers are added such that no carries occur between stages, then the add time is simply the propagation time through a single full-adder.

4- Bit Parallel Subtractor:

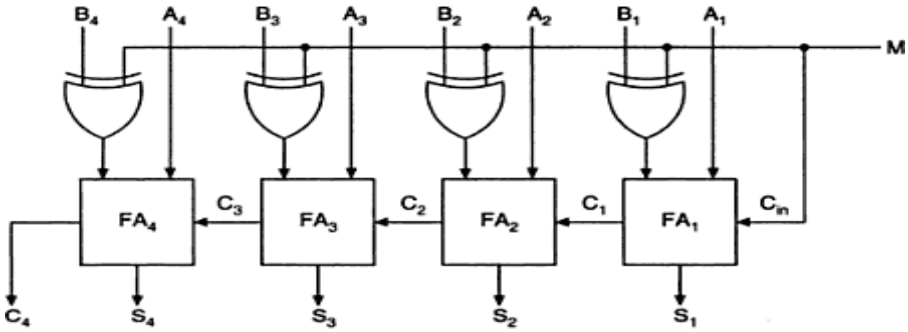
The subtraction of binary numbers can be carried out most conveniently by means of complements, the subtraction A-B can be done by taking the 2‘s complement of B and adding it to A . The 2‘s complement can be obtained by taking the 1‘s complement and adding 1 to the least significant pair of bits. The 1‘s complement can be implemented with inverters as shown below in the diagram.



Logic diagram of a 4-bit parallel subtractor.

Binary-Adder Subtractor:

A 4-bit adder-subtractor, the addition and subtraction operations are combined into one circuit with one common binary adder. This is done by including an X-OR gate with each full-adder. The mode input M controls the operation. When M=0, the circuit is an adder, and when M=1, the circuit becomes a subtractor. Each X-OR gate receives input M and one of the inputs of B. When M=0, $B \oplus 0 = B$. The full-adder receives the value of B, the input carry is 0 and the circuit performs A+B. when $B \oplus 1 = B'$ and C1=1. The B inputs are complemented and a 1 is through the input carry. The circuit performs the operation A plus the 2‘s complement of B.



Logic diagram of a 4-bit binary adder-subtractor.

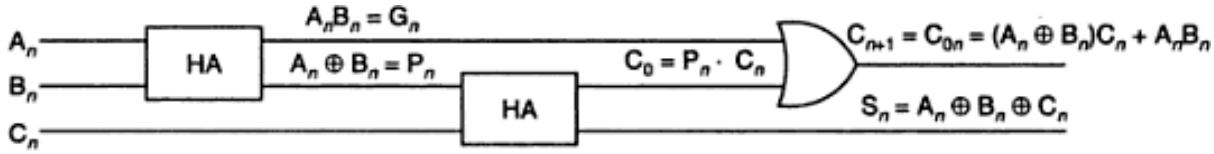
The Look-Ahead –Carry Adder:

In parallel-adder, the speed with which an addition can be performed is governed by the time required for the carries to propagate or ripple through all of the stages of the adder. The look-ahead carry adder speeds up the process by eliminating this ripple carry delay. It examines all the input bits simultaneously and also generates the carry-in bits for all the stages simultaneously.

The method of speeding up the addition process is based on the two additional functions of the full-adder, called the carry generate and carry propagate functions.

Consider one full adder stage; say the nth stage of a parallel adder as shown in fig. we know that is made by two half adders and that the half adder contains an X-OR gate to produce the sum and an AND gate to produce the carry. If both the bits An and Bn are 1s, a carry must be generated in

this stage regardless of whether the input carry Cin is a 0 or a 1. This is called generated carry, expressed as Gn= An.Bn which has to appear at the output through the OR gate as shown in fig.



A full adder (*n*th stage of a parallel adder).

There is another possibility of producing a carry out. X-OR gate inside the half-adder at the input produces an intermediary sum bit- call it Pn –which is expressed as $P_n = A_n \oplus B_n$. Next Pn and Cn are added using the X-OR gate inside the second half adder to produce the final

sum bit and $S_n = P_n \oplus C_n$ where $P_n = A_n \oplus B_n$ and output carryC0= Pn. Cn=($A_n \oplus B_n$)Cn which becomes carry for the (n+1)th stage.

Consider the case of both Pn and Cn being 1. The input carry Cn has to be propagated to the output only if Pn is 1. If Pn is 0, even if Cn is 1, the and gate in the second half-adder will inhibit Cn . the carry out of the nth stage is 1 when either Gn=1 or Pn.Cn =1 or both Gn and Pn.Cn are equal to 1.
For the final sum and carry outputs of the nth stage, we get the following Boolean expressions.

$$S_n = P_n \oplus C_n \text{ where } P_n = A_n \oplus B_n$$
$$C_{on} = C_{n+1} = G_n + P_nC_n \text{ where } G_n = A_n \cdot B_n$$

Observe the recursive nature of the expression for the output carry at the nth stage which becomes the input carry for the (n+1)st stage .it is possible to express the output carry of a higher significant stage is the carry-out of the previous stage.

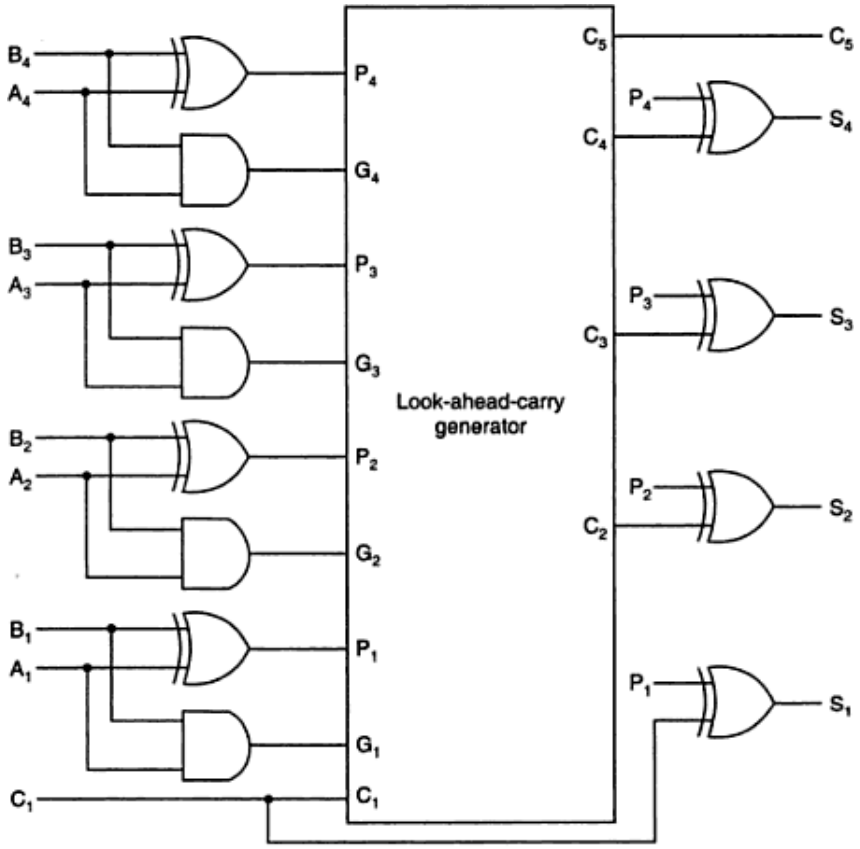
Based on these, the expression for the carry-outs of various full adders are as follows,

$$C_1 = G_0 + P_0 \cdot C_0$$
$$C_2 = G_1 + P_1 \cdot C_1 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$
$$C_3 = G_2 + P_2 \cdot C_2 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$
$$C_4 = G_3 + P_3 \cdot C_3 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

The general expression for *n* stages designated as 0 through (*n* – 1) would be

$$C_n = G_{n-1} + P_{n-1} \cdot C_{n-1} = G_{n-1} + P_{n-1} \cdot G_{n-2} + P_{n-1} \cdot P_{n-2} \cdot G_{n-3} + \ldots + P_{n-1} \cdot \ldots P_0 \cdot C_0$$

Observe that the final output carry is expressed as a function of the input variables in SOP form. Which is two level AND-OR or equivalent NAND-NAND form. Observe that the full look-ahead scheme requires the use of OR gate with (n+1) inputs and AND gates with number of inputs varying from 2 to (n+1).



Logic diagram of a 4-bit look-ahead-carry adder.

2's complement Addition and Subtraction using Parallel Adders:

Most modern computers use the 2's complement system to represent negative numbers and to perform subtraction operations of signed numbers that can be performed using only the addition operation, if we use the 2's complement form to represent negative numbers.
The circuit shown can perform both addition and subtraction in the 2's complement. This adder/subtractor circuit is controlled by the control signal ADD/SUB'. When the ADD/SUB' level is HIGH, the circuit performs the addition of the numbers stored in registers A and B. When the ADD/Sub' level is LOW, the circuit subtract the number in register B from the number in register A. The operation is:

When ADD/SUB' is a 1:

1. AND gates 1,3,5 and 7 are enabled , allowing B0,B1,B2and B3 to pass to the OR gates 9,10,11,12 . AND gates 2,4,6 and 8 are disabled , blocking B0',B1',B2', and B3' from reaching the OR gates 9,10,11 and 12.
2. The two levels B0 to B3 pass through the OR gates to the 4-bit parallel adder, to be added to the bits A0 to A3. The sum appears at the output S0 to S3
3. Add/SUB' =1 causes no carry into the adder.

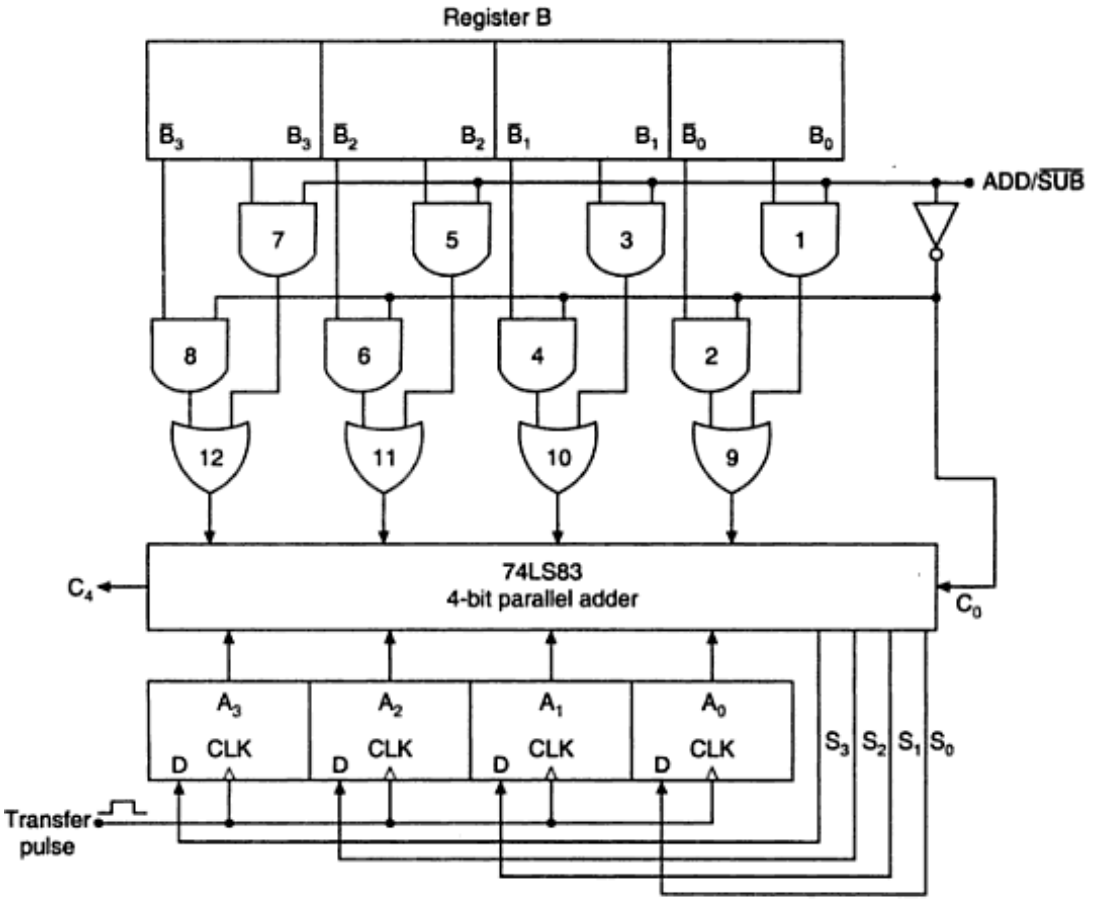
When ADD/SUB‘ is a 0:

1. AND gates 1,3,5 and 7 are disabled, allowing B0,B1,B2and B3 from reaching the OR gates 9,10,11,12 . AND gates 2,4,6 and 8 are enabled, blocking B0‘,B1‘,B2‘, and B3‘ from reaching the OR gates.

2. The two levels B0‘ to B3‘ pass through the OR gates to the 4-bit parallel adder, to be added to the bits A0 to A3.The C0 is now 1.thus the number in register B is converted to its 2‘s complement form.

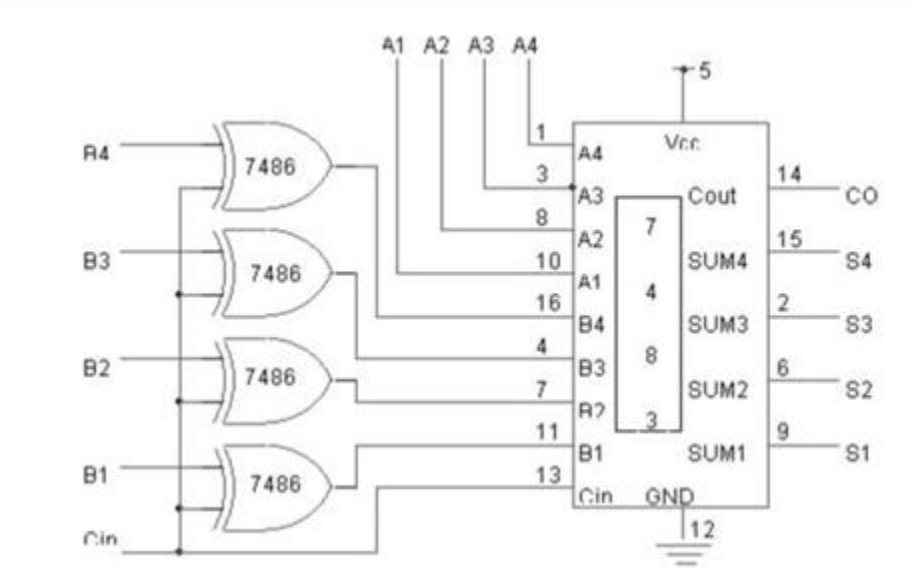
3. The difference appears at the output S0 toS3.

Adders/Subtractors used for adding and subtracting signed binary numbers. In computers, the output is transferred into the register A (accumulator) so that the result of the addition or subtraction always end up stored in the register A This is accomplished by applying a transfer pulse to the CLK inputs of register A.



Logic diagram of a parallel adder/subtractor using 2's complement system.

Simplified diagram of 4-bit Parallel Adder/Subtractor using IC-7483:

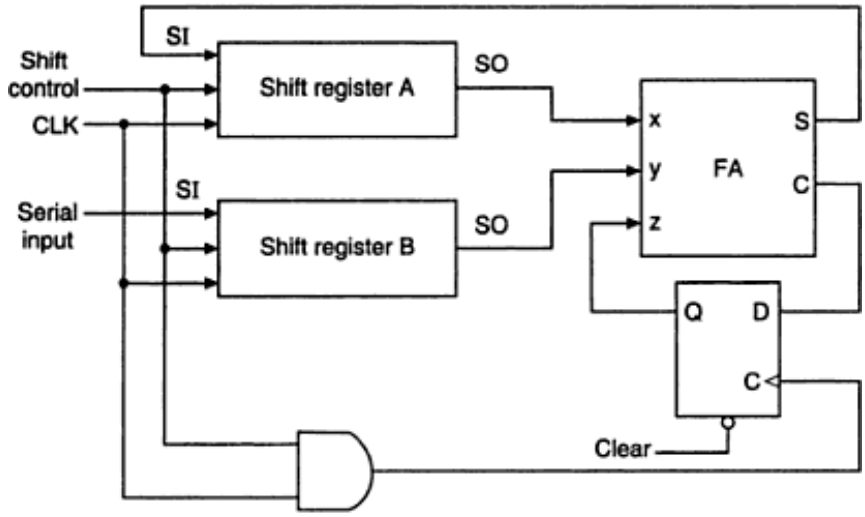


Serial Adder:

A serial adder is used to add binary numbers in serial form. The two binary numbers to be added serially are stored in two shift registers A and B. Bits are added one pair at a time through a single full adder (FA) circuit as shown. The carry out of the full-adder is transferred to a D flip-flop. The output of this flip-flop is then used as the carry input for the next pair of significant bits. The sum bit from the S output of the full-adder could be transferred to a third shift register. By shifting the sum into A while the bits of A are shifted out, it is possible to use one register for storing both augend and the sum bits. The serial input register B can be used to transfer a new binary number while the addend bits are shifted out during the addition.
The operation of the serial adder is:
Initially register A holds the augend, register B holds the addend and the carry flip-flop is cleared to 0. The outputs (SO) of A and B provide a pair of significant bits for the full-adder at x and y. The shift control enables both registers and carry flip-flop, so, at the clock pulse both registers are shifted once to the right, the sum bit from S enters the left most flip-flop of A, and the output carry is transferred into flip-flop Q. The shift control enables the registers for a number of clock pulses equal to the number of bits of the registers. For each succeeding clock pulse a new sum bit is transferred to A, a new carry is transferred to Q, and both registers are shifted once to the right. This process continues until the shift control is disabled. Thus the addition is accomplished by passing each pair of bits together with the previous carry through a single full adder circuit and transferring the sum, one bit at a time, into register A.

Initially, register A and the carry flip-flop are cleared to 0 and then the first number is added from B. While B is shifted through the full adder, a second number is transferred to it through its serial input. The second number is then added to the content of register A while a third number is

transferred serially into register B. This can be repeated to form the addition of two, three, or more numbers and accumulate their sum in register A.



Logic diagram of a serial adder.

1. Add the 4-bit BCD code groups for each decimal digit position using ordinary binary addition.

2. For those positions where the sum is 9 or less, the sum is in proper BCD form and no correction is needed.

3. When the sum of two digits is greater than 9, a correction of 0110 should be added to that sum, to produce the proper BCD result. This will produce a carry to be added to the next decimal position.

Difference between Serial and Parallel Adders:

The parallel adder registers with parallel load, whereas the serial adder uses shift registers. The number of full adder circuits in the parallel adder is equal to the number of bits in the binary numbers, whereas the serial adder requires only one full adder circuit and a carry flip- flop. Excluding the registers, the parallel adder is a combinational circuit, whereas the serial adder is a sequential circuit. The sequential circuit in the serial adder consists of a full-adder and a flip-flop that stores the output carry.

BCD Adder:

The BCD addition process:
A BCD adder circuit must be able to operate in accordance with the above steps. In other words, the circuit must be able to do the following:
1. Add two 4-bit BCD code groups, using straight binary addition.

2. Determine, if the sum of this addition is greater than 1101 (decimal 9); if it is, add 0110 (decimal 6) to this sum and generate a carry to the next decimal position.

The first requirement is easily met by using a 4- bit binary parallel adder such as the 74LS83 IC. For example, if the two BCD code groups A3A2A1A0 and B3B2B1B0 are applied to a 4-bit parallel adder, the adder will output S4S3S2S1S0, where S4 is actually C4, the carry –out of the MSB bits.

The sum outputs S4S3S2S1S0 can range anywhere from 00000 to 10010 (when both the BCD code groups are 1001=9). The circuitry for a BCD adder must include the logic needed to detect whenever the sum is greater than 01001, so that the correction can be added in. Those cases, where the sum is greater than 1001 are listed as:

S_4	S_3	S_2	S_1	S_0	Decimal number
0	1	0	1	0	10
0	1	0	1	1	11
0	1	1	0	0	12
0	1	1	0	1	13
0	1	1	1	0	14
0	1	1	1	1	15
1	0	0	0	0	16
1	0	0	0	1	17
1	0	0	1	0	18

Let us define a logic output X that will go HIGH only when the sum is greater than 01001 (i.e, for the cases in table). If examine these cases, see that X will be HIGH for either of the following conditions:

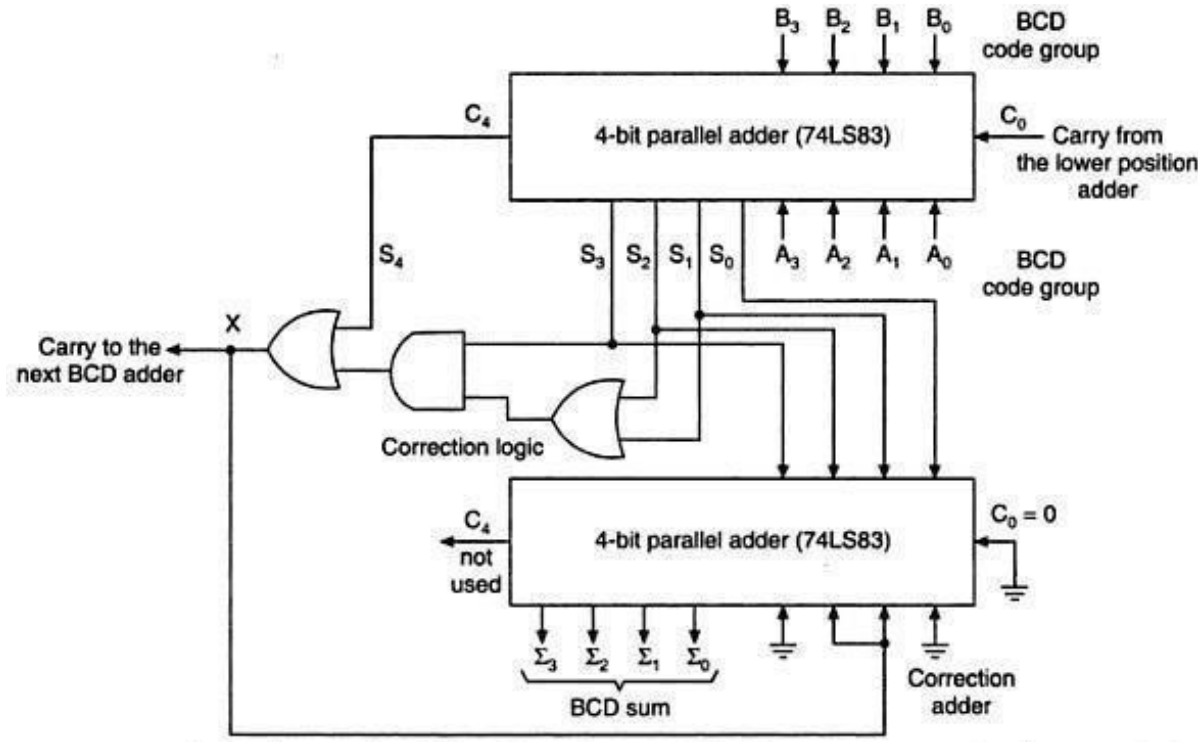
1. Whenever S4 =1(sum greater than15)

2. Whenever S3 =1 and either S2 or S1 or both are 1 (sum 10 to 15) This condition can be expressed as

$$X=S4+S3(S2+S1)$$

Whenever X=1, it is necessary to add the correction factor 0110 to the sum bits, and to generate a carry. The circuit consists of three basic parts. The two BCD code groups A3A2A1A0 and B3B2B1B0 are added together in the upper 4-bit adder, to produce the sum S4S3S2S1S0. The logic gates shown implement the expression for X. The lower 4-bit adder will add the correction 0110 to the sum bits, only when X=1, producing the final BCD sum output represented by Σ3Σ2Σ1Σ0. The X is also the carry-out that is produced when the sum is greater than 01001. When X=0, there is no carry and no addition of 0110. In such cases, Σ3Σ2Σ1Σ0= S3S2S1S0.

Two or more BCD adders can be connected in cascade when two or more digit decimal numbers are to be added. The carry-out of the first BCD adder is connected as the carry-in of the second BCD adder, the carry-out of the second BCD adder is connected as the carry-in of the third BCD adder and so on.



Logic diagram of a BCD adder using two 4-bit adders and a correction-detector circuit.

EXCESS-3(XS-3) ADDER:

To perform Excess-3 additions,
1. Add two xs-3 code groups
2. If carry=1, add 0011(3) to the sum of those two code groups
3. If carry =0, subtract 0011(3) i.e., add 1101 (13 in decimal) to the sum of those two code groups.

Ex: Add 9 and 5

(a)

	1100	**9 in Xs-3**
	+1000	**5 in Xs-3**
	0100	**there is a carry**
1		
+0011	**0011**	**add 3 to each**
----------	**----------**	**group**
0100	**0111**	**14 in xs-3**
(1)	**(4)**	

Ex: Add 4 and 3

```
(b)            0 1 1 1    4 in XS-3
             + 0 1 1 0    3 in XS-3
             -----------
               1 1 0 1    no carry
             + 1 1 0 1    Subtract 3 (i.e. add 13)
             -----------
Ignore carry 1 1 0 1 0    7 in XS-3
                (7)
```

Implementation of xs-3 adder using 4-bit binary adders is shown. The augend (A3 A2A1A0) and addend (B3B2B1B0) in xs-3 are added using the 4-bit parallel adder. If the carry is a 1, then 0011(3) is added to the sum bits S3S2S1S0 of the upper adder in the lower 4-bit parallel adder. If the carry is a 0, then 1101(3) is added to the sum bits (This is equivalent to subtracting 0011(3) from the sum bits. The correct sum in xs-3 is obtained.

Excess-3 (XS-3) Subtractor:

To perform Excess-3 subtraction,
1. Complement the subtrahend
2. Add the complemented subtrahend to the minuend.
3. If carry =1, result is positive. Add 3 and end around carry to the result. If carry=0, the result is negative. Subtract 3, i.e, and take the 1‘s complement of the result. (1) 0100 There is a carry

```
Ex:    Perform 9-4
        1100          9 in xs-3
       +1000          Complement of 4 n Xs-3
        --------
(1)     0100          There is a carry
       +0011          Add 0011(3)
      ------------
        0111
           1          End around carry
      ------------
        1000          5 in xs-3
```

The minuend and the 1‘s complement of the subtrahend in xs-3 are added in the upper 4- bit parallel adder. If the carry-out from the upper adder is a 0, then 1101 is added to the sum bits of the upper adder in the lower adder and the sum bits of the lower adder are complemented to get the result. If the carry-out from the upper adder is a 1, then 3=0011 is added to the sum bits of the lower adder and the sum bits of the lower adder give the result.

Decoders

The decoders are used to decode binary information in a digital system. The input lines to a decoder are used to feed binary data while based on the binary data, one of the output lines is

selected which represents the respective minterm. If a decoder has n input lines then, it can have maximum 2^n output lines. With n number of input lines, the decoder can decode n-bit digital data. Like with 3 input lines, a decoder can decode 3-bit binary data and so maximum 2^3 or 8 binary symbols. If there are some unused combinations in n-bit coded information, the decoder can have fewer than 2^n output lines. So, any decoder is n-to-m line decoder where $m <= 2^n$.

In a decoder, each output line represents a minterm which is selected based on the input binary data. For any input data only one output line is activated at a time. It can be noted that in a decoder, the number of output lines is always greater than number of input lines. If the number of inputs and outputs in a decoder circuit are equal, it is then called converter.

The decoders are usually constructed using AND or NAND Gates. The output of an AND gate is only HIGH when all the inputs are HIGH. So, AND gate is the basic decoding element in a decoder circuit. For example, in a 4-to-16-line decoder, the digital circuit used in decoding the binary code 1001 will be as follow –

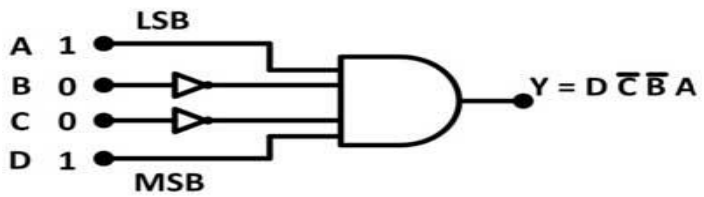
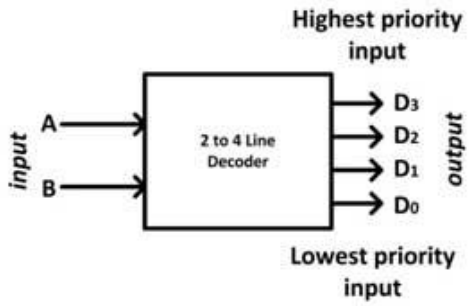


Image showing Decoding Circuit Built using AND Gate

In the decoding element (of the 4-to-16-line decoder) shown above, to make all the inputs of the AND gate HIGH, the two middle bits must be inverted by using two NOT gates. So, when the input binary data will be 1001, the only output line connected to the above decoding circuit will be activated and set HIGH.

For simplicity let us take 2-to-4-line decoder as an example. A 2-to-4-line decoder has two input line (say A and B). With 2 inputs, 4 binary codes can be represented, so there can be 4 output lines (say D0, D1, D2 and D3). Each output line represents a minterm like D0 represents A'B', D1 represents A'B, D2 represents AB' and D3 represents AB. A 2-to-4-line decoder can be represented by the following block diagram –

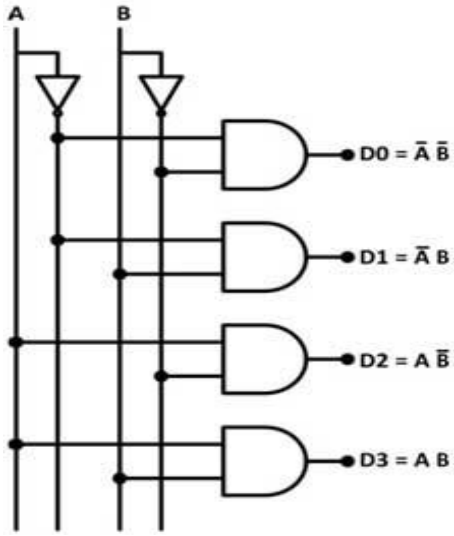


Block Diagram of 2-to-4-Line Decoder

It will have the following truth table –

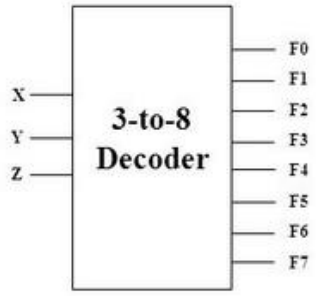
INPUTS		OUTPUTS			
A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
0	1	0	0	1	0
1	1	0	0	0	1

From the above truth table, the digital circuit for 2-to-4-line decoder can be constructed using AND gates and NOT gates as follow –

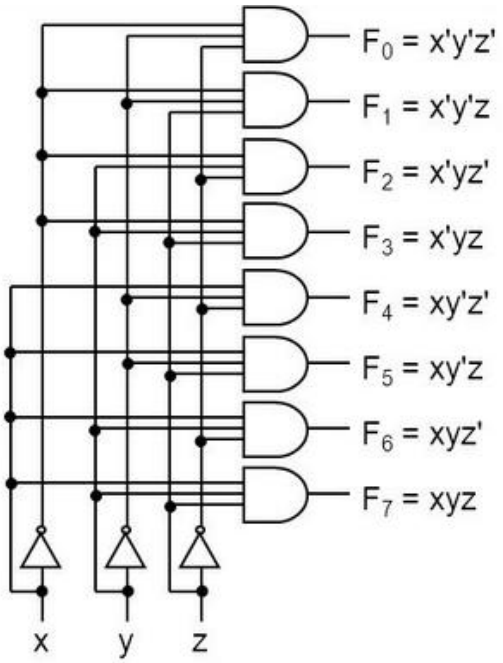


Circuit Diagram of 2-to-4-Line Decoder

A 3:8 Decoder:



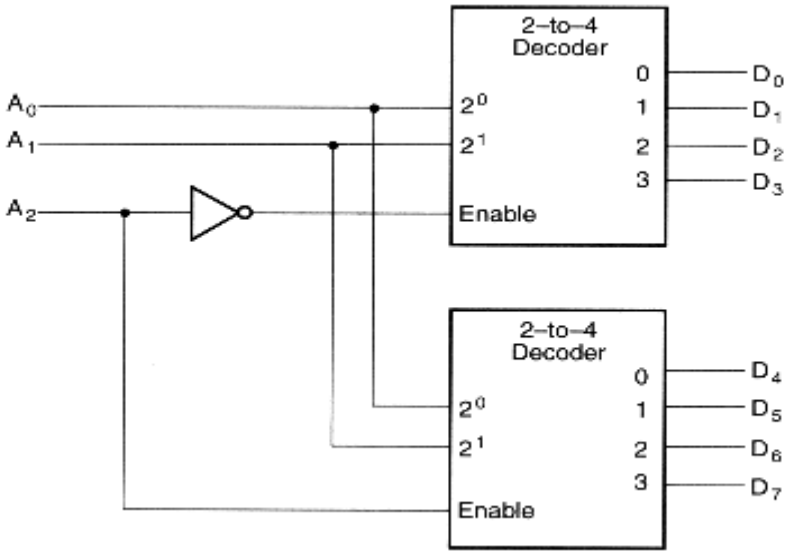
x	y	z	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



Logic function Implementation:

Realization of 3:8 decoder using two 2:4 decoder.

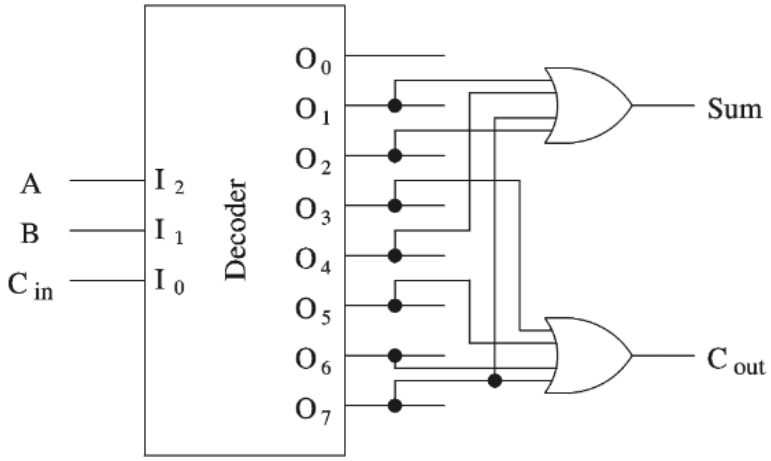
Two to four decoder design with its *enable* inputs can be used to build a three to eight decoder as below.



Similarly, 4:16 decoder can be designed with two 3:8 decoders and 5:32 also could be designed using four 3:8 decoder and one 2:4 decoder.

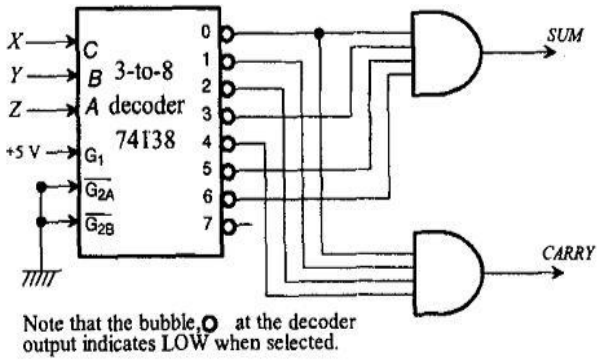
Realization of Full adder-using 2x4 Decoder IC-74139:

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



In the above circuit, *S* is given by [SUM] *m*(1, 2, 4, 7) while *C* is given by [SUM] *m*(3, 5, 6, 7) as given by the minterms each of the OR gates are connected to.

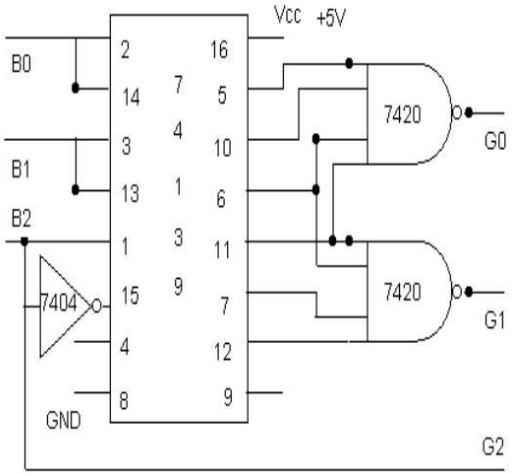
Realization of Full adder using the 3x8 Decoder IC-74138:



Implementation of a Full-adder Using a 74138 Decoder and Two 4-input AND Gates

Realization of 3-bit Binary to Gray code converter using IC-74139:

BINARY			GRAY		
B_2	B_1	B_0	G_2	G_1	G_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0



Similarly other logic circuitary can be designed by using different Decoder ICs.

Decoder Applications:

Microprocessor memory systems

–selecting different banks of memory

Microprocessor input/output systems

–selecting different devices

Microprocessor instruction decoding

–enabling different functional units

Memory chips

–enabling different rows of memory depending on address

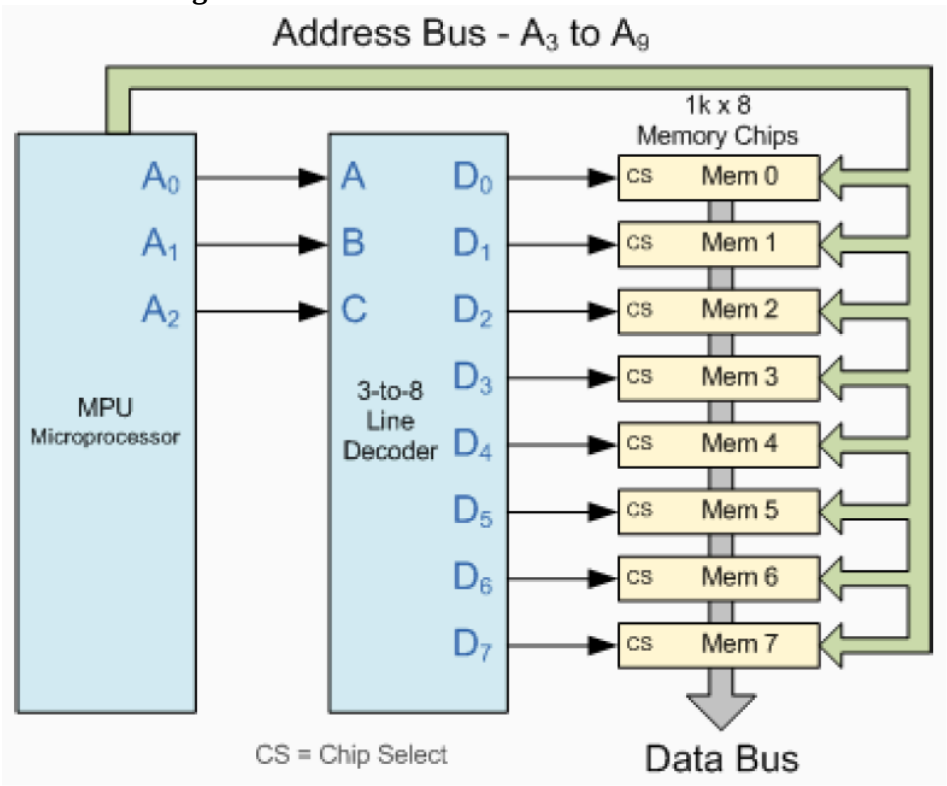
One application of Decoder:

Memory Address Decoder.

In modern microprocessor systems the amount of memory required can be quite high and is generally more than one single memory chip alone. One method of overcoming this problem is to connect lots of individual memory chips together and to read the data on a common "Data Bus". In order to prevent the data being "read" from each memory chip at the same time, each memory chip is selected individually one at time and this process is known as Address Decoding. Each memory chip has an input called Chip Select or CS which is used by the MPU to select the appropriate memory chip and a logic "1" on this input selects the device and a logic "0" on the input de-selects it. By selecting or de-selecting each chip, allows us to select the correct memory device for a particular address and when we specify a particular memory address, the corresponding memory location exists ONLY in one of the chips.

For example, Lets assume we have a very simple microprocessor system with only 1Kb of RAM memory and 10 address lines. The memory consists of 128x8-bit (128x8 = 1024 bytes) devices and for 1Kb we will need 8 individual memory devices but in order to select the correct memory chip we will also require a 3-to-8 line binary decoder as shown below.

Memory Address Decoding.



The binary decoder requires 3 address lines, (A0 to A2) to select each one of the 8 chips (the lower part of the address), while the remaining 7 address lines (A3 to A9) select the correct memory location on that chip (the upper part of the address). Having selected a memory location using the address bus, the information at the particular internal memory location is sent to the "Data Bus" for use by the microprocessor. This is of course a simple example but the principle remains the same for all types of memory chips or modules.

Binary Decoders:

Binary Decoders are very useful devices for converting one digital format to another, such as binary or BCD type data into decimal or octal etc and commonly available decoder IC's are the TTL 74LS138 3-to-8 line binary decoder or the 74ALS154 4-to-16 line decoder. They are also very useful for interfacing to 7-segment displays such as the TTL 74LS47 which we will look at in the next tutorial.

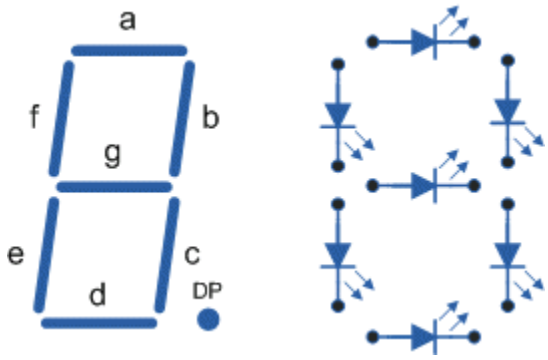
Display Decoders

A Decoder IC, is a device which converts one digital format into another and the most commonly used device for doing this is the BCD (Binary Coded Decimal) to 7-Segment Display Decoder. 7-segment LED (Light Emitting Diode) or LCD (Liquid Crystal) Displays, provide a very convenient way of displaying information or digital data in the form of Numbers, Letters or even Alpha-numerical characters and they consist of 7 individual LEDs (the segments), within one single display package. In order to produce the required numbers or characters from 0 to 9 and A to F respectively, on the display the correct combination of LED segments need to be illuminated and Display Decoders do just that. A standard 7-segment LED or LCD display generally has 8 input connections, one for each LED segment and one that acts as a common terminal or connection for all the internal segments. Some single displays have an additional input pin for the decimal point in their lower right or left hand corner.

There are two important types of 7-segment LED digital display.

- ✓ The Common Cathode Display (CCD) - In the common cathode display, all the cathode connections of the LEDs are joined together to logic "0" and the individual segments are illuminated by application of a "HIGH", logic "1" signal to the individual Anode terminals.
- ✓ The Common Anode Display (CAD) - In the common anode display, all the anode connections of the LEDs are joined together to logic "1" and the individual segments are illuminated by connecting the individual Cathode terminals to a "LOW", logic "0" signal.

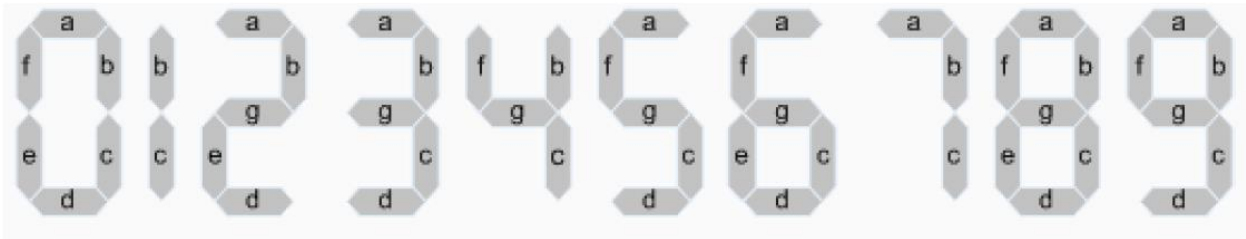
7-Segment Display Format



Truth Table for 7-Segment Display:

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	0	1	1	1	1	1	6
7	0	1	1	1	1	1	1	0	0	0	0	7
8	1	0	0	0	1	1	1	1	1	1	1	8
9	1	0	0	1	1	1	1	1	0	1	1	9

The Pattern of Numbers:



It can be seen that to display any single digit number from 0 to 9 or letter from A to F, we would need 7 separate segment connections plus one additional connection for the LED's "common" connection. Also as the segments are basically a standard light emitting diode, the driving circuit would need to produce up to 20mA of current to illuminate each individual segment and to display the number 8, all 7 segments would need to be lit resulting a total current of nearly 140mA, (8 x 20mA). Obviously, the use of so many connections and power consumption is impractical for some electronic or microprocessor based circuits and so in order to reduce the number of signal lines required to drive just one single display, display decoders such as the BCD to 7-Segment Display Decoder and Driver IC's are used instead.

Binary Coded Decimal:

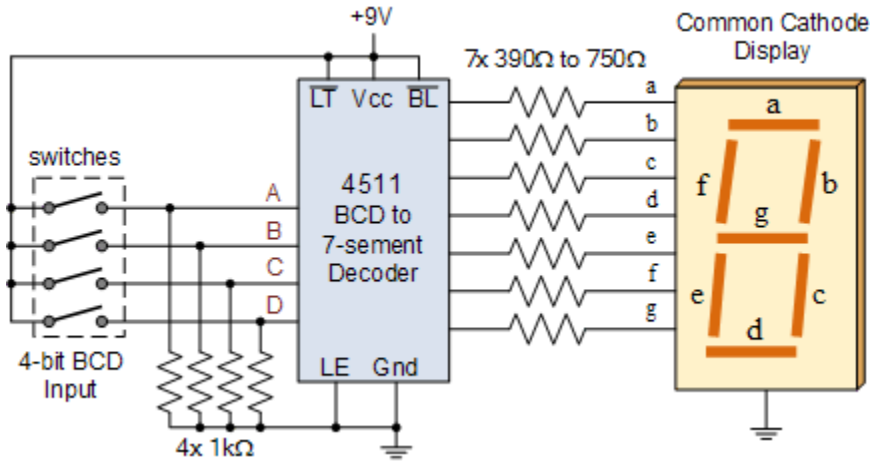
Binary Coded Decimal (BCD or "8421" BCD) numbers are made up using just 4 data bits (a nibble or half a byte) similar to the Hexadecimal numbers we saw in the binary tutorial, but unlike hexadecimal numbers that range in full from 0 through to F, BCD numbers only range from 0 to 9, with the binary number patterns of 1010 through to 1111 (A to F) being invalid inputs for this type of display and so are not used as shown below.

Decimal	Binary Pattern				BCD	Decimal	Binary Pattern				BCD
	8	4	2	1			8	4	2	1	
0	0	0	0	0	0	8	1	0	0	0	8
1	0	0	0	1	1	9	1	0	0	1	9
2	0	0	1	0	2	10	1	0	1	0	Invalid
3	0	0	1	1	3	11	1	0	1	1	Invalid
4	0	1	0	0	4	12	1	1	0	0	Invalid
5	0	1	0	1	5	13	1	1	0	1	Invalid
6	0	1	1	0	6	14	1	1	1	0	Invalid
7	0	1	1	1	7	15	1	1	1	1	Invalid

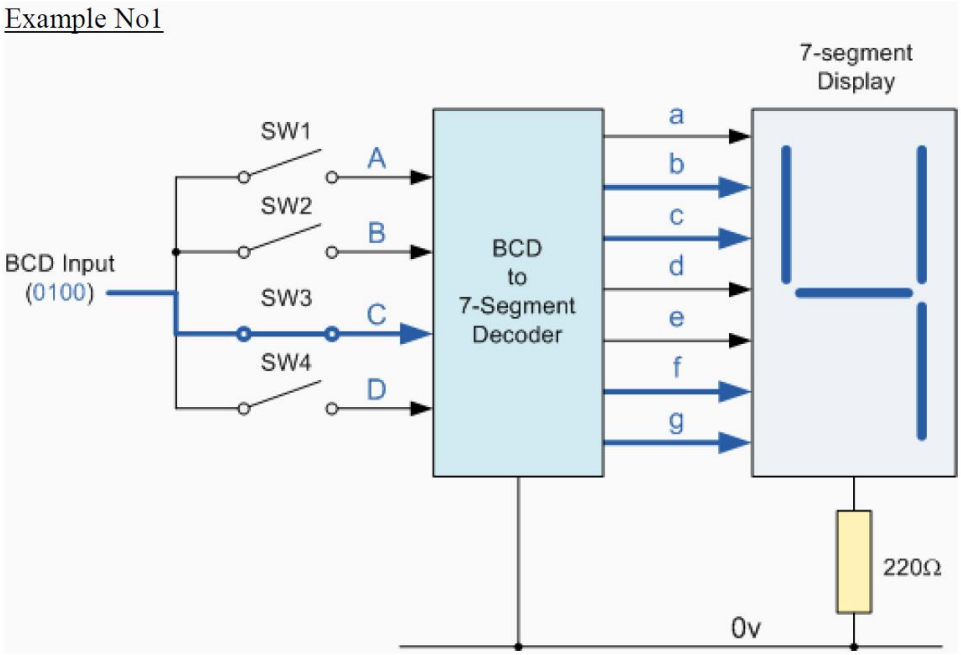
BCD to 7-Segment Display Decoders:

A binary coded decimal (BCD) to 7-segment display decoder such as the TTL 74LS47 or 74LS48, have 4 BCD inputs and 7 output lines, one for each LED segment. This allows a smaller 4-bit binary number (half a byte) to be used to display all the denary numbers from 0 to 9 and by adding two displays together, a full range of numbers from 00 to 99 can be displayed with just a single byte of 8 data bits.

BCD to 7-Segment Decoder:



The use of packed BCD allows two BCD digits to be stored within a single byte (8-bits) of data, allowing a single data byte to hold a BCD number in the range of 00 to 99.
An example of the 4-bit BCD input (0100) representing the number 4 is given below.



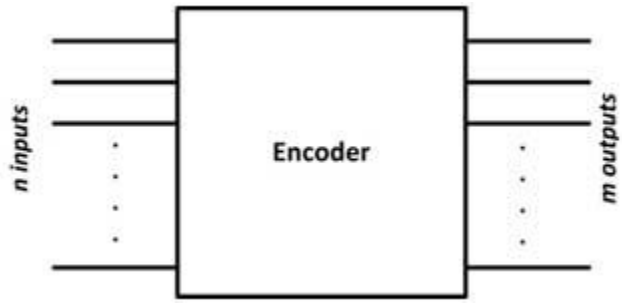
In practice current limiting resistors of about 150Ω to 220Ω would be connected in series between the decoder/driver chip and each LED display segment to limit the maximum current flow. Different display decoders or drivers are available for the different types of display available, e.g. 74LS48 for common-cathode LED types, 74LS47 for common-anode LED types, or the CMOS CD4543 for liquid crystal display (LCD) types.

Liquid crystal displays (LCD´s) have one major advantage over similar LED types in that they consume much less power and nowadays, both LCD and LED displays are combined together to form larger Dot-Matrix Alphanumeric type displays which can show letters and characters as well as numbers in standard Red or Tri-colour outputs.

Encoders

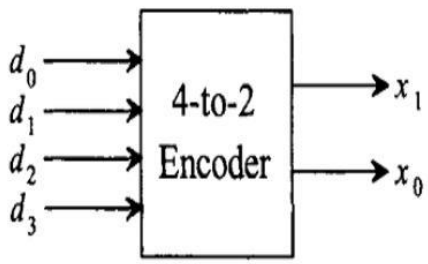
The encoders are used for coding binary data. They are opposite to decoders. So, they perform inverse operation of a decoder. The opposite of the decoding process is called encoding. The encoder is a combinational logic circuit that converts an active input signal into a coded output signal.

Generally encoders produce outputs of 2-bit, 3-bit or 4-bit codes depending upon the number of data input lines and a "n-bit" encoder has 2n input lines with common types that include 4-to-2, 8-to-3 and 16-to-4 line configurations. Encoders are available to encode either a decimal or hexadecimal input pattern to typically a binary or B.C.D. output code. An encoder can be represented by the following block diagram -



Block Diagram of Encoder

A 4:2 Encoder:

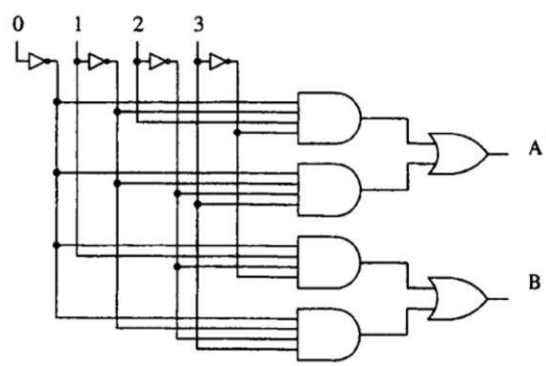


Truth Table of the 4-to-2 Encoder

Inputs				Outputs	
d_0	d_1	d_2	d_3	x_1	x_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

In the encoder defined by table, it is assumed that only one of the four inputs can be HIGH at any time. If more than one input is 1 at the same time, an undefined output is generated. For example, if d_3, and d_2 are 1 at the same time, both x0 and x1 are 1. This represents binary 3 rather than 1 or 2. One simple way to overcome this problem is to "Prioritize" the level of each input pin and if there was more than one input at logic level "1" the actual output code would only correspond to the input with the highest designated priority. Then this type of encoder is known as Priority Encoders or P-encoder.

	Decimal				Binary	
	0	1	2	3	*A*	*B*
0	1	0	0	0	0	0
1	0	1	0	0	0	1
2	0	0	1	0	1	0
3	0	0	0	1	1	1



Logic diagram of 4-2 Encoder

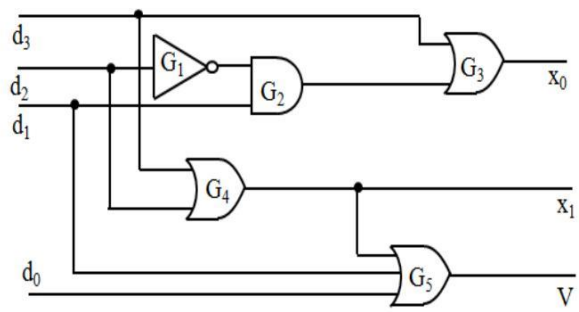
(a) Priority Encoder:

A 4-to-2 priority encoder may be designed assuming that inputs with higher subscripts have higher priorities. This means that d3 has the highest priority and d0 has the lowest priority. Therefore, if d0 and d3, become one simultaneously, the output will be 11. The truth table of the 4-to-2 priority encoder is shown in Figure 1.61 shows the function table, logic diagram and the K-maps of the 4-to-2 priority encoder.

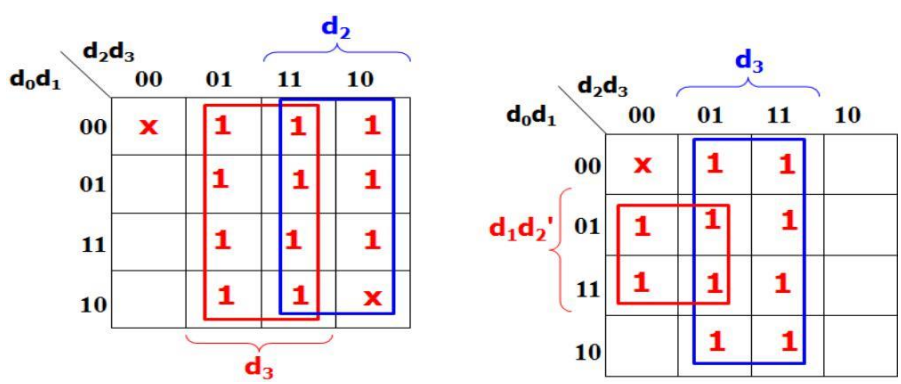
Truth Table of the 4-to-2 Priority Encoder

Inputs				Outputs	
d_0	d_1	d_2	d_3	x_1	x_0
1	0	0	0	0	0
X	1	0	0	0	1
X	X	1	0	1	0
X	X	X	1	1	1

(a) Function table



(b) Logic diagram

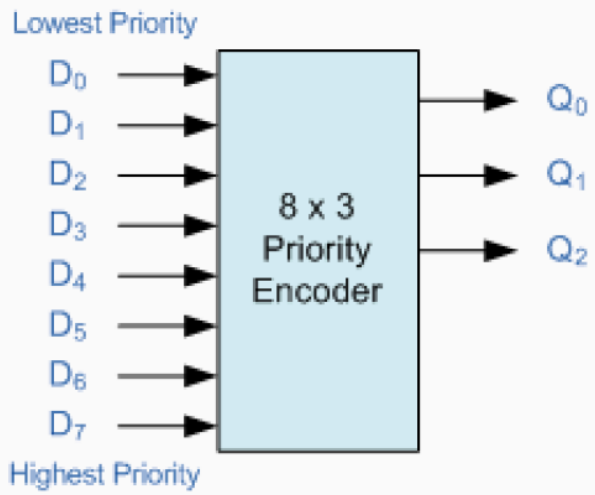


(c) K-maps

Output Expressions:

x1 = d2 + d3 x0 = d3 + d3 d2“ V = d0 + d1 + d2 + d3

8:3 Priority Encoder:



Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	x	0	0	1
0	0	0	0	0	1	x	x	0	1	0
0	0	0	0	1	x	x	x	0	1	1
0	0	0	1	x	x	x	x	1	0	0
0	0	1	x	x	x	x	x	1	0	1
0	1	x	x	x	x	x	x	1	1	0
1	x	x	x	x	x	x	x	1	1	1

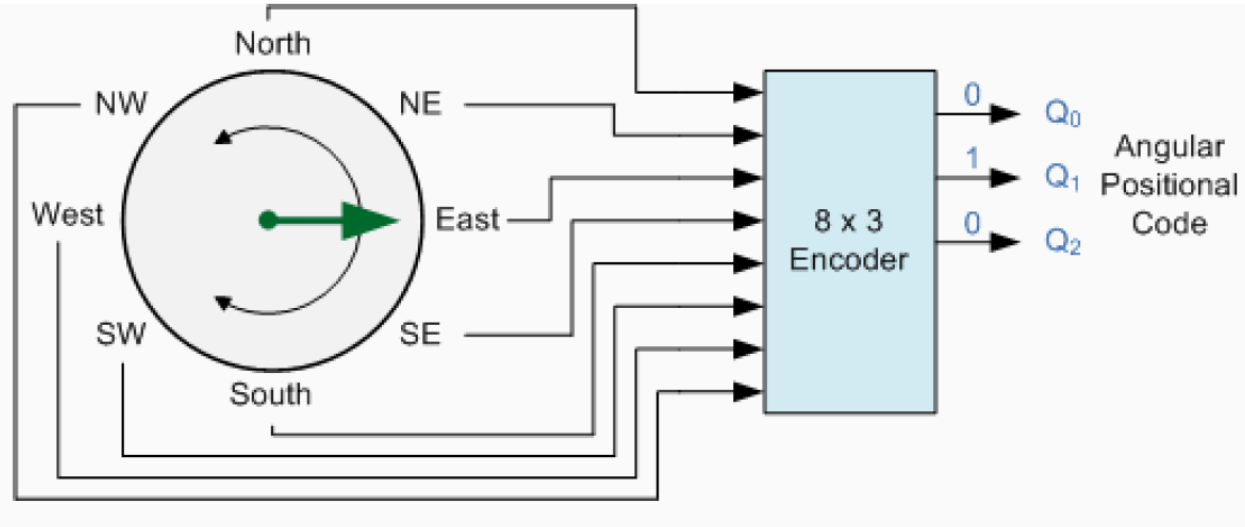
X = dont care

Priority encoders are available in standard IC form and the TTL 74LS148 is an 8 to 3 bit priority encoder which has eight active LOW (logic "0") inputs and provides a 3-bit code of the highest ranked input at its output. Priority encoders output the highest order input first for example, if input lines "D2", "D3" and "D5" are applied simultaneously the output code would be for input "D5" ("101") as this has the highest order out of the 3 inputs. Once input "D5" had been removed the next highest output code would be for input "D3" ("011"), and so on.

Encoder Applications:
Keyboard Encoders:

Priority encoders can be used to reduce the number of wires needed in circuits or applications that has multiple inputs. For example, assume that a microcomputer needs to read the 104 keys of a standard QWERTY keyboard where only one key would be pressed or HIGH at any one time. One way would be to connect all 104 wires from the keys directly to the computer but this would be impractical for a small home PC, but another better way would be to use an encoder. The 104 individual buttons or keys could be encoded into a standard ASCII code of only 7-bits (0 to 127 decimal) to represent each key or character and then inputted as a much smaller 7-bit B.C.D code directly to the computer. Keypad encoders such as the 74C923 20-key encoder are available.

Positional Encoders:
Another more common application is in magnetic positional control as used on ships or robots etc. Here the angular or rotary position of a compass is converted into a digital code by an encoder and inputted to the systems computer to provide navigational data and an example of a simple 8 position to 3-bit output compass encoder is shown below.



Compass Direction	Binary Output		
	Q_0	Q_1	Q_2
North	0	0	0
North-East	0	0	1
East	0	1	0
South-East	0	1	1
South	1	0	0
South-West	1	0	1
West	1	1	0
North-West	1	1	1

Interrupt Requests :

Other applications for Priority Encoders may include detecting interrupts in microprocessor applications. Here the microprocessor uses interrupts to allow peripheral devices such as the disk drive, scanner, mouse, or printer etc, to communicate with it, but the microprocessor can only "talk" to one peripheral device at a time. The processor uses "Interrupt Requests" or "IRQ" signals to assign priority to the devices to ensure that the most important peripheral device is serviced first. The order of importance of the devices will depend upon their connection to the priority encoder.

Because implementing such a system using priority encoders such as the standard 74LS148 priority encoder IC involves additional logic circuits, purpose built integrated circuits such as the 8259 Programmable Priority Interrupt Controller is available.

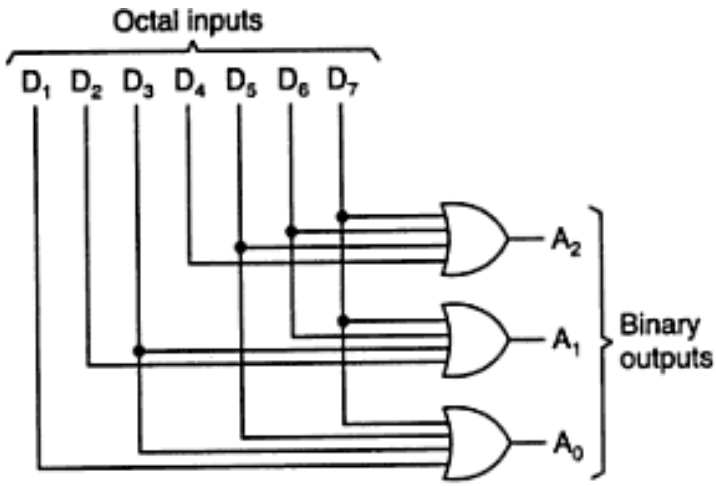
IRQ Number	Typical Use	Description
IRQ 0	System timer	Internal System Timer.
IRQ 1	Keyboard	Keyboard Controller.
IRQ 3	COM2 & COM4	Second and Fourth Serial Port.
IRQ 4	COM1 & COM3	First and Third Serial Port.
IRQ 5	Sound	Sound Card.
IRQ 6	Floppy disk	Floppy Disk Controller.
IRQ 7	Parallel port	Parallel Printer.
IRQ 12	Mouse	PS/2 Mouse.
IRQ 14	Primary IDE	Primary Hard Disk Controller.
IRQ 15	Secondary IDE	Secondary Hard Disk Controller.

Realization of combinational logic circuits using Encoders:

Octal to Binary Encoder:

Octal digits		Binary		
		A_2	A_1	A_0
D_0	0	0	0	0
D_1	1	0	0	1
D_2	2	0	1	0
D_3	3	0	1	1
D_4	4	1	0	0
D_5	5	1	0	1
D_6	6	1	1	0
D_7	7	1	1	1

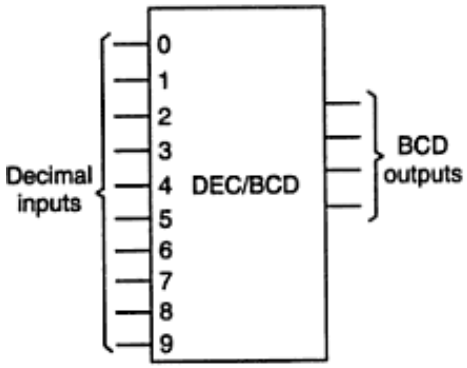
(a) Truth table



(b) Logic diagram

Octal-to-binary encoder.

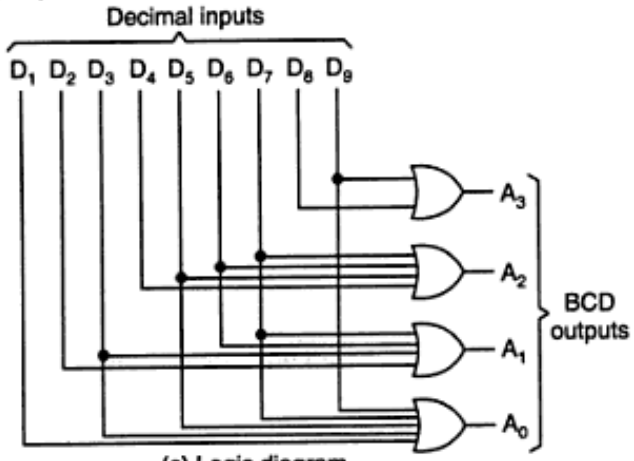
Decimal to BCD Encoder:



(a) Logic symbol

Decimal inputs		Binary A_3	A_2	A_1	A_0
D_0	0	0	0	0	0
D_1	1	0	0	0	1
D_2	2	0	0	1	0
D_3	3	0	0	1	1
D_4	4	0	1	0	0
D_5	5	0	1	0	1
D_6	6	0	1	1	0
D_7	7	0	1	1	1
D_8	8	1	0	0	0
D_9	9	1	0	0	1

(b) Truth table

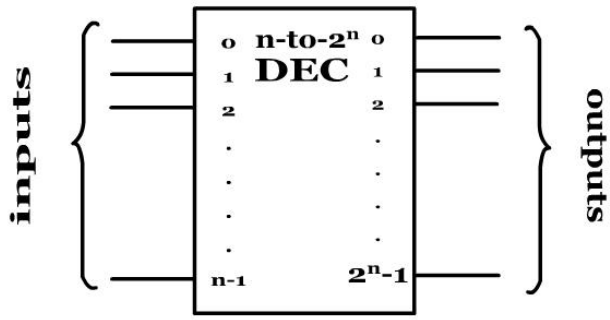


(c) Logic diagram

Decimal-to-BCD encoder.

Decoder

A Decoder is a multiple input, multiple output logic circuit. The block diagram of a decoder is as shown below.

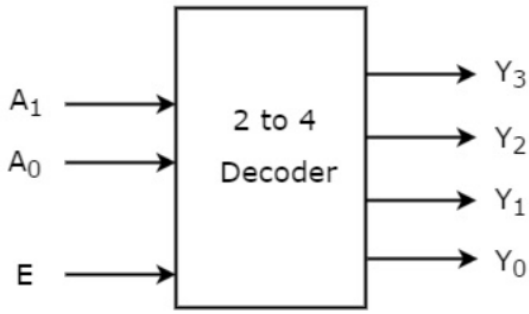


An n-to-2^nline decoder symbol.

The most commonly used decoder is a n –to 2^n decoder which has n inputs and 2^n Output lines.

2 to 4 Decoder

Let 2 to 4 Decoder has two inputs A_1 & A_0 and four outputs Y_3, Y_2, Y_1 & Y_0. The **block diagram** of 2 to 4 decoder is shown in the following figure.



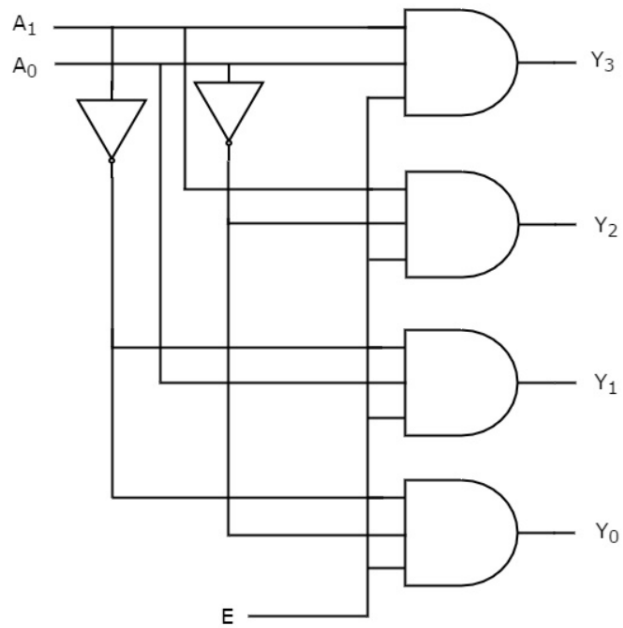
One of these four outputs will be '1' for each combination of inputs when enable, E is '1'. The **Truth table** of 2 to 4 decoder is shown below.

Enable	Inputs		Outputs			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

From Truth table, we can write the **Boolean functions** for each output as

$$Y3=E.A1.A0$$
$$Y2=E.A1.A0'$$
$$Y1=E.A1'.A0$$
$$Y0=E.A1'.A0'$$

Each output is having one product term. So, there are four product terms in total. We can implement these four product terms by using four AND gates having three inputs each & two inverters. The **circuit diagram** of 2 to 4 decoder is shown in the following figure.



Therefore, the outputs of 2 to 4 decoder are nothing but the **min terms** of two input variables A_1 & A_0, when enable, E is equal to one. If enable, E is zero, then all the outputs of decoder will be equal to zero.

Similarly, 3 to 8 decoder produces eight min terms of three input variables A_2, A_1 & A_0 and 4 to 16 decoder produces sixteen min terms of four input variables A_3, A_2, A_1 & A_0.

Implementation of Higher-order Decoders

Now, let us implement the following two higher-order decoders using lower-order decoders.

- 3 to 8 decoder
- 4 to 16 decoder

3 to 8 Decoder

In this section, let us implement **3 to 8 decoder using 2 to 4 decoders**. We know that 2 to 4 Decoder has two inputs, A_1 & A_0 and four outputs, Y_3 to Y_0. Whereas, 3 to 8 Decoder has three inputs A_2, A_1 & A_0 and eight outputs, Y_7 to Y_0.

We can find the number of lower order decoders required for implementing higher order decoder using the following formula.

$$Required\ number\ of\ lower\ order\ decoders = m_2 / m_1$$

Where,

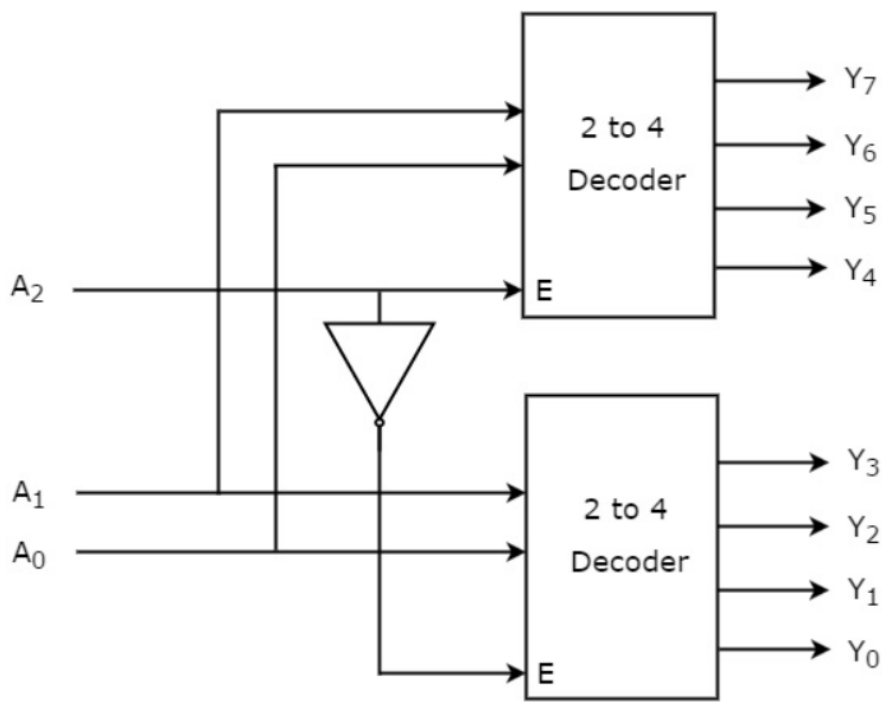
m_1 is the number of outputs of lower order decoder.

m_2 is the number of outputs of higher order decoder.

Here, $m_1 = 4$ and $m_2 = 8$. Substitute, these two values in the above formula.

$$Required\ number\ of\ 2\ to\ 4\ decoders = 8/4 = 2$$

Therefore, we require two 2 to 4 decoders for implementing one 3 to 8 decoder. The **block diagram** of 3 to 8 decoder using 2 to 4 decoders is shown in the following figure.



The parallel inputs A_1 & A_0 are applied to each 2 to 4 decoder. The complement of input A_2 is connected to Enable, E of lower 2 to 4 decoder in order to get the outputs, Y_3 to Y_0. These are the **lower four min terms**. The input, A_2 is directly connected to Enable, E of upper 2 to 4 decoder in order to get the outputs, Y_7 to Y_4. These are the **higher four min terms**.

4 to 16 Decoder

In this section, let us implement **4 to 16 decoder using 3 to 8 decoders**. We know that 3 to 8 Decoder has three inputs A_2, A_1 & A_0 and eight outputs, Y_7 to Y_0. Whereas, 4 to 16 Decoder has four inputs A_3, A_2, A_1 & A_0 and sixteen outputs, Y_{15} to Y_0

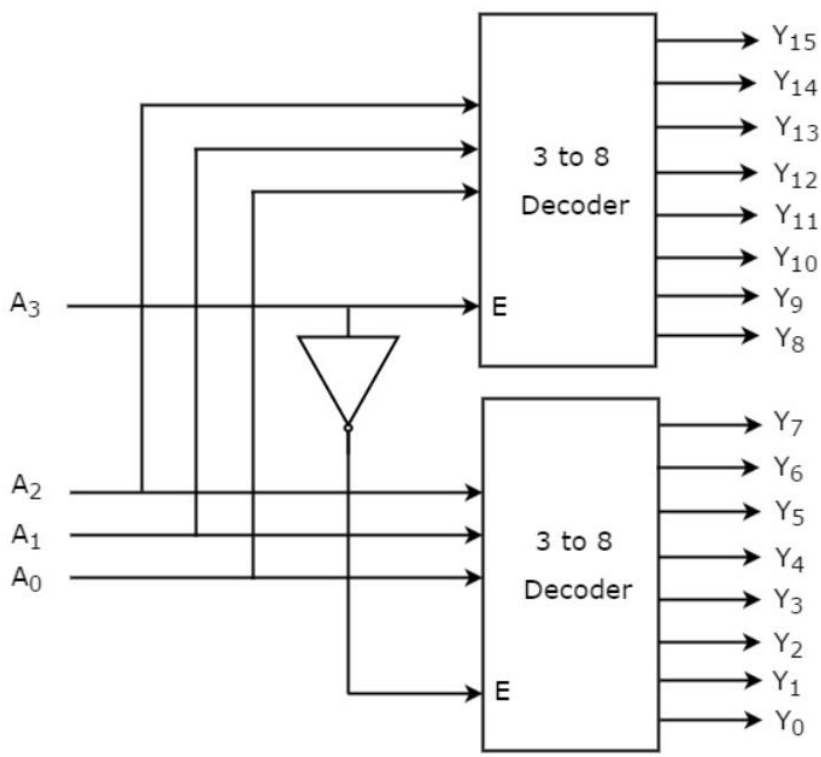
We know the following formula for finding the number of lower order decoders required.

$$\textit{Required number of lower order decoders} = m2/\,m1$$

Substitute, $m1 = 8$ and $m2 = 16$ in the above formula.

$$\textit{Required number of } 3to8 \textit{ decoders} = 16/8 = 2$$

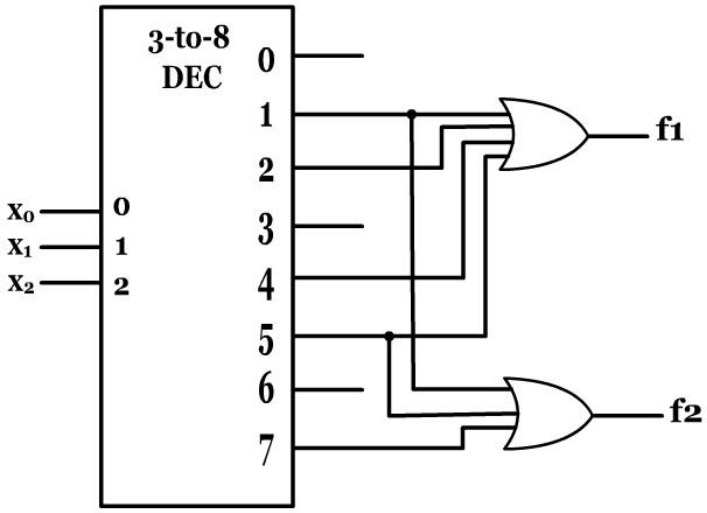
Therefore, we require two 3 to 8 decoders for implementing one 4 to 16 decoder. The **block diagram** of 4 to 16 decoder using 3 to 8 decoders is shown in the following figure.



The parallel inputs A_2, A_1 & A_0 are applied to each 3 to 8 decoder. The complement of input, A3 is connected to Enable, E of lower 3 to 8 decoder in order to get the outputs, Y_7 to Y_0. These are the **lower eight min terms**. The input, A_3 is directly connected to Enable, E of upper 3 to 8 decoder in order to get the outputs, Y_{15} to Y_8. These are the **higher eight min terms**.

Realization of Boolean expression using Decoder and OR gate:

We see from the above truth table that the output expressions correspond to a single minterm. Hence a n –to 2_n decoder is a minterm generator. Thus, by using OR gates in conjunction with n to 2^n decoder boolean function realization is possible.
Ex: to realize the Boolean functions given below using decoders…
•F1=Σm(1,2,4,5)
•F2=Σm(1,5,7)

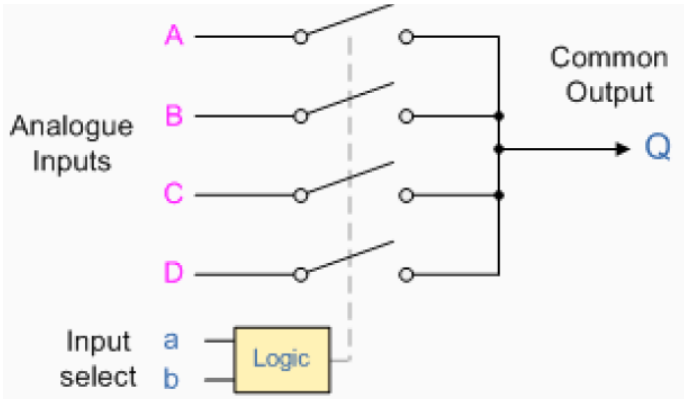


Multiplexers and De-Multiplexers

The Multiplexers:

The Multiplexer which sometimes are simply called "Mux" or "Muxes", are devices that act like a very fast acting rotary switch. They connect multiple input lines 2, 4, 8, 16 etc one at a time to a common output line and are used as one method of reducing the number of logic gates required in a circuit. Multiplexers are individual Analogue Switches as opposed to the "mechanical" types such as normal conventional switches and relays. They are usually made from MOSFETs devices encased in a single package and are controlled using standard logic gates. An example of a Multiplexer is shown below.

4-to-1 Channel Multiplexer



Addressing		Input Selected
b	a	
0	0	A
0	1	B
1	0	C
1	1	D

The Boolean expression for this 4 to 1 **Multiplexer** is given as:

$$Q = a'b'A + a'bB + ab'C + abD$$

In this example at any instant in time only one of the four analogue switches is closed, connecting only one of the input lines A to D to the single output at Q. As to which switch is closed depends upon the addressing input code on lines "a" and "b", so for this example to select input B to the output at Q, the binary input address would need to be "a" = logic "1" and "b" = logic "0". Adding more control address lines will allow the multiplexer to control more inputs. Multiplexers can also be used to switch either analogue, digital or video signals, with the switching current in analogue circuits limited to below 10mA to 20mA per channel in order to reduce heat dissipation.

Hence a multiplexer,

Also called Data selector

A digital circuit which selects one of the n data inputs and route it to the single output.

Select lines (n) and Input lines are (2n)

Multiplexer acts like a multiple way switch

•The output gets connected to only one of the n data inputs at given instant of time.

E is enable input

–Useful for cascading

–Its an active low terminal

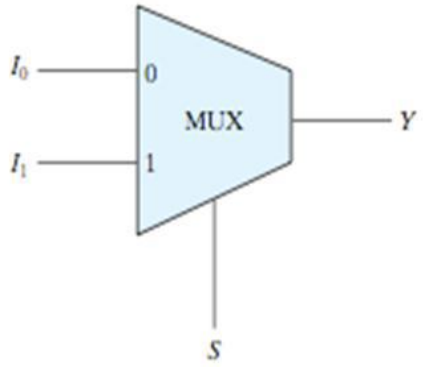
–It will perform the required operation when it is low

Necessity of Multiplexer:

- ✓ In most of the digital circuits, data is available on more than 1 line
- ✓ It is essential to route this data over a single line
- ✓ It reduces the number of wires, complexity, cost

Classification of Multiplexer:

a) 2:1 multiplexer
b) 4:1 multiplexer
c) 8:1 multiplexer and d)16:1 multiplexer and follows

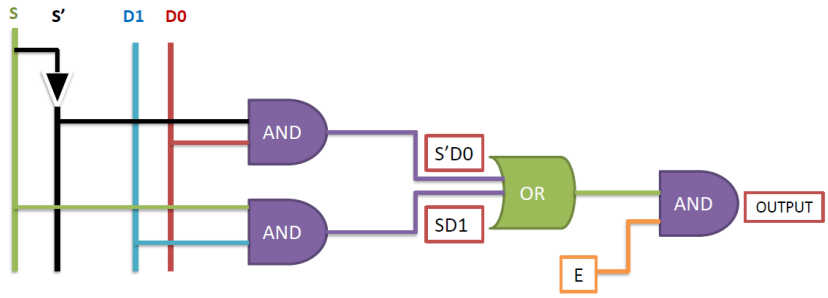


2 : 1 Multiplexer:

Data inputs are 2 (D0,D1)

Select input is 1 (S)

Enable	S	Output
0	X	
1	0	D0
1	1	D1



Functional Table *Logic Circuit*

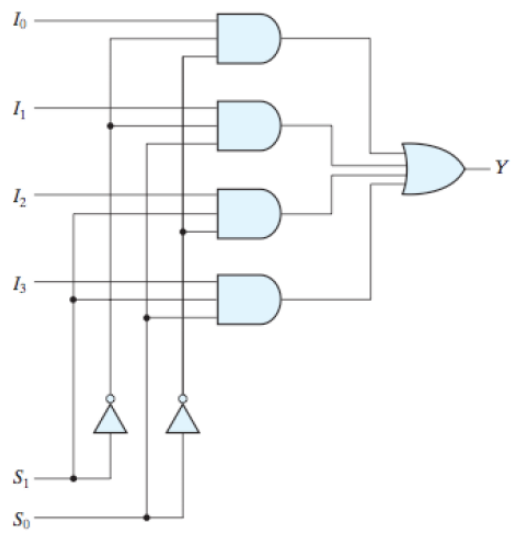
4 : 1 Multiplexer

Data inputs are 4 (I0-I3)

Select input are 2 (S1 , S2)

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Functional Table

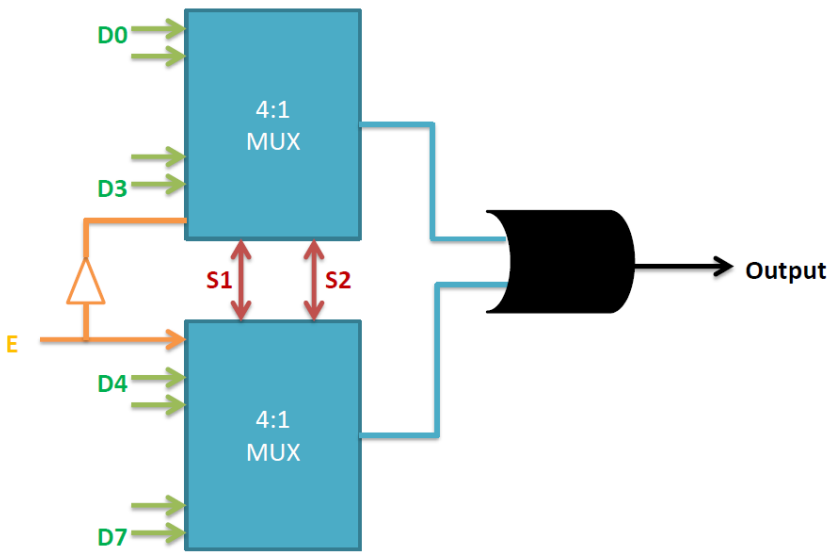


Logic Circuit

Similarly, 8:1 and 16:1 mux can also be designed.

Cascading of Multiplexers:

Obtain 8:1 MUX using 4:1 MUXES



Multiplexers as General-purpose Logic:

2^n:1 multiplexer implements any function of n variables

–With the variables used as control inputs and

–Data inputs tied to 0 or 1

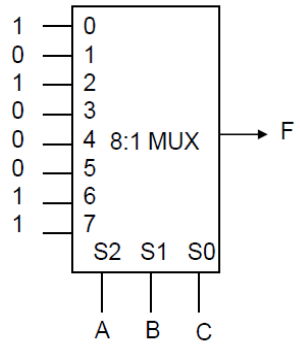
–In essence, *a lookup table*

•Example:

–F(A,B,C) = m0 + m2 + m6 + m7

= A'B'C' + A'BC' + ABC' + ABC

= A'B'(C') + A'B(C') + AB'(0) + AB(1)



2n-1:1 mux can implement any function of n variables

–With n-1 variables used as control inputs and

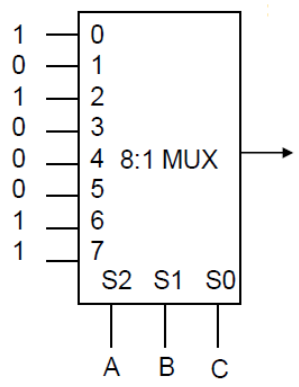
–Data inputs tied to the last variable or its complement

•Example:

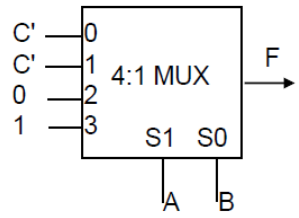
–F(A,B,C) = m0 + m2 + m6 + m7

= A'B'C' + A'BC' + ABC' + ABC

= A'B'(C') + A'B(C') + AB'(0) + AB(1)

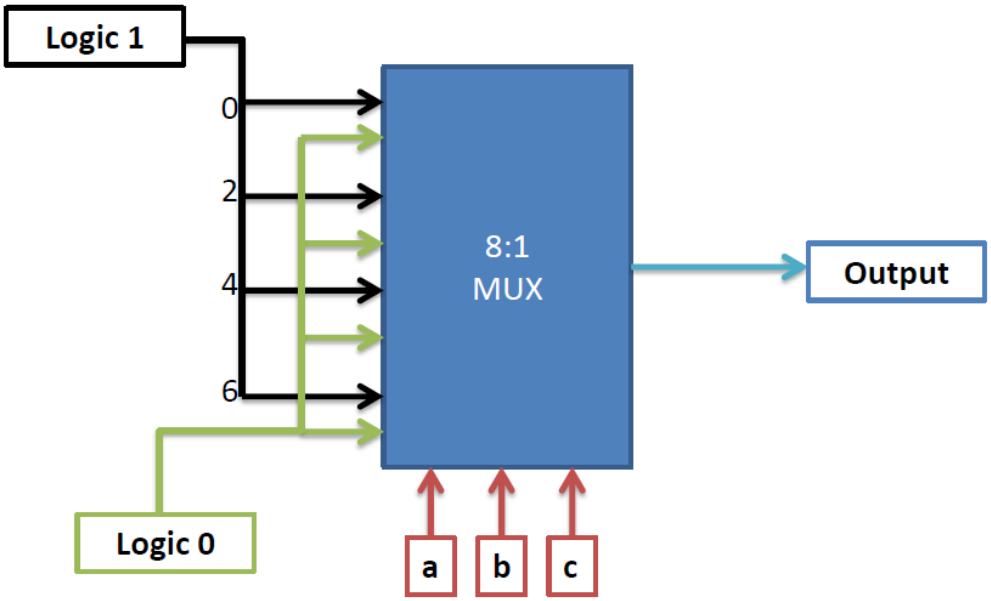


A	B	C	F	
0	0	0	1	C'
0	0	1	0	
0	1	0	1	C'
0	1	1	0	
1	0	0	0	0
1	0	1	0	
1	1	0	1	1
1	1	1	1	

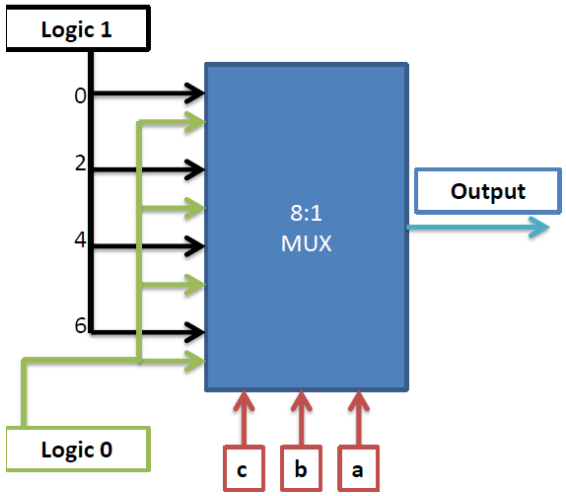


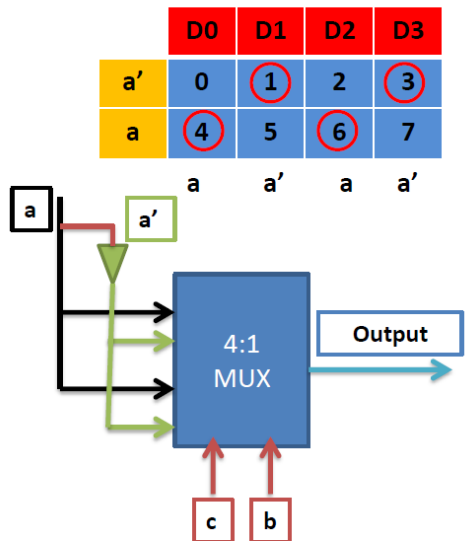
Example-1:

Implement the following expression using 8:1 MUX F (a,b,c) = m (0,2,4,6)



Example-2:
Implement the logic function using 4:1 MUX F (a,b,c) = Σm (1,3,4,6)

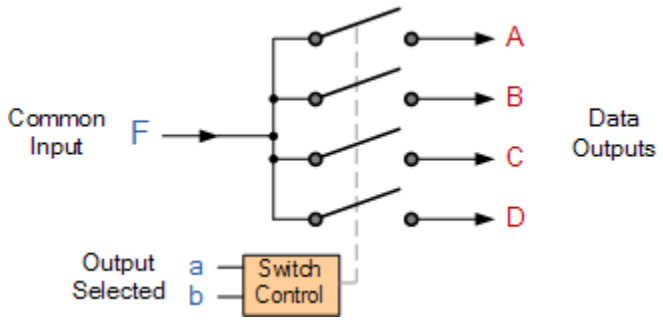




The Demultiplexer

The data distributor, known more commonly as a Demultiplexer or "Demux" for short, is the exact opposite of the Multiplexer. The demultiplexer takes one single input data line and then switches it to any one of a number of individual output lines one at a time. The demultiplexer converts a serial data signal at the input to a parallel data at its output lines as shown below.

1-to-4 Channel De-multiplexer:



Truth Table:

Output Select		Data Output Selected
b	a	
0	0	A

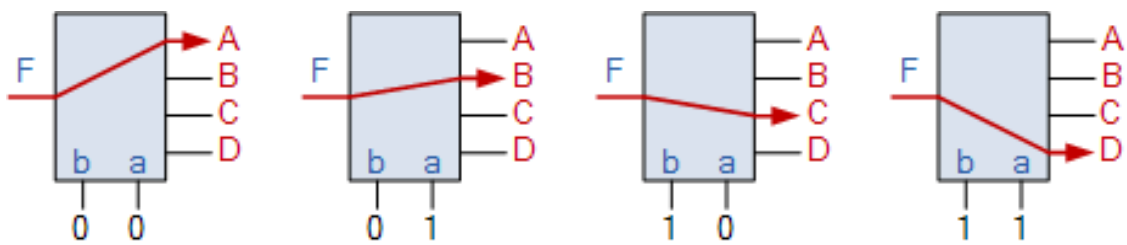
0	1	B
1	0	C
1	1	D

The Boolean expression for this 1-to-4 **Demultiplexer** above with outputs A to D and data select lines a, b is given as:

$$F = abA + abB + abC + abD$$

The function of the **Demultiplexer** is to switch one common data input line to any one of the 4 output data lines A to D in our example above. As with the multiplexer the individual solid state switches are selected by the binary input address code on the output select pins "a" and "b" as shown.

Demultiplexer Output Line Selection:

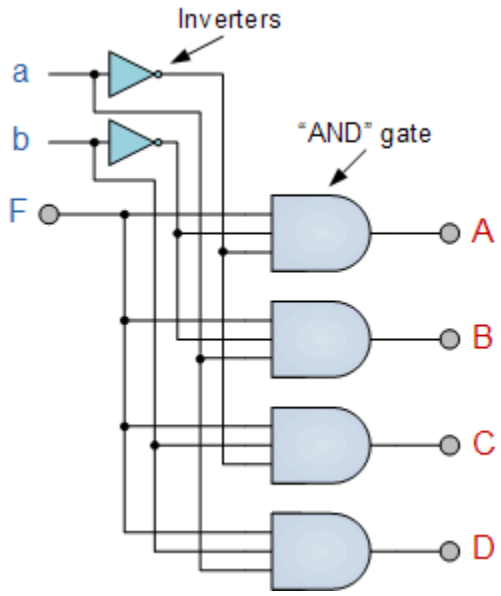


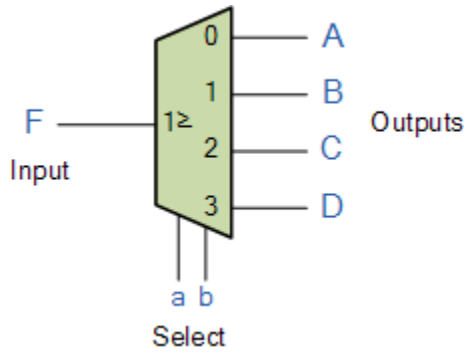
As with the previous multiplexer circuit, adding more address line inputs it is possible to switch more outputs giving a 1-to-2^n data line outputs.

Some standard demultiplexer IC´s also have an additional "enable output" pin which disables or prevents the input from being passed to the selected output. Also some have latches built into their outputs to maintain the output logic level after the address inputs have been changed. However, in standard decoder type circuits the address input will determine which single data output will have the same value as the data input with all other data outputs having the value of logic "0".

The implementation of the Boolean expression above using individual logic gates would require the use of six individual gates consisting of AND and NOT gates as shown.

4 Channel Demultiplexer using Logic Gates:



The symbol used in logic diagrams to identify a demultiplexer is as follows.

The Demultiplexer Symbol: